

Time Series Forecasting

Government data of rainfall during monsoon around Mahanadi river was provided to predict floods

The work is divided into 5 sections:

Prerequisite

1. Learning Python and Its Libraries

My journey began with self-learning the basics of Python, including fundamental programming concepts such as variables, loops, functions, and object-oriented programming. To enhance my skills, I also explored widely-used libraries:

- **Pandas:** Used for data manipulation and analysis. I learned to handle structured datasets, clean data, and perform operations like filtering, grouping, and aggregating.
- **NumPy:** Focused on numerical computing, enabling efficient operations on large multi-dimensional arrays and matrices.
- **Matplotlib:** Helped with data visualization, allowing you to create various plots such as line charts, histograms, and scatter plots to understand data patterns.

2. Exploring Deep Learning: TensorFlow & PyTorch

Then I moved on to deep learning frameworks, specifically TensorFlow and PyTorch, which are widely used for building neural networks and machine learning models.

- **TensorFlow:** It provides computational efficiency for large-scale deep learning models.
- **PyTorch:** It offers dynamic computational graphs, making debugging and experimentation more flexible.

Converting MLP Transformer from TensorFlow to PyTorch

This required an in-depth understanding of both frameworks and neural network architectures.

Project Breakdown

a) Defining Functions for Data Preprocessing

Before training a model, data preprocessing is essential to ensure that inputs are clean and properly structured. This involved:

- Handling missing values.
- Normalizing or standardizing data.
- Encoding categorical variables.
- Splitting datasets into different sections.

b) Implementing the Transformer Model

A transformer model, originally introduced in NLP tasks, relies on self-attention mechanisms to process input data. In this step:

- I structured the layers of the transformer model.
- Defined forward propagation for feature transformation.
- Integrated activation functions like ReLU (Rectified Linear Unit) to introduce non-linearity.

c) Splitting the Dataset

To properly train and evaluate the model, I divided the dataset into three parts:

- **Training set:** Used to train the model on patterns within the data.
- **Validation set:** Used to tune hyperparameters and avoid overfitting.
- **Testing set:** Evaluated the final model's performance on unseen data.

d) Defining Hyperparameters

- **Learning rate:** Controls how much the model updates during each step.
- **Batch size:** Determines the number of samples processed before updating weights.
- **Number of epochs:** Specifies how many times the model goes through the entire dataset.
- **Optimizer selection:** Choosing methods like Adam, SGD (Stochastic Gradient Descent), or RMSprop to update model weights efficiently.

e) Model Training and Optimization

After defining the model and hyperparameters, I trained the model by:

- Feeding input data into the network.
- Computing loss using a metric like Mean Squared Error (MSE) or Cross-Entropy Loss.
- Backpropagating the errors to adjust model weights.

- Continuously optimizing predictions to achieve better accuracy.

f) Performance Evaluation Metrics

Once training was complete, I evaluated the model's effectiveness using multiple performance metrics:

1. Nash-Sutcliffe Efficiency (NSE):

- Measures how well the model's predictions match the actual data.
- A value closer to 1 indicates high accuracy, while values near 0 or negative imply poor predictions.

2. Evolution Criterion (Evol):

- Likely used to assess how the model improves over training epochs.
- Helps determine convergence and whether further training is required.

3. Total Percent Error (TPE):

- Calculates the percentage difference between predicted and actual values.
- Lower values indicate better model performance.

Converting MLP Layer to KAN Layer

1. Understanding the Need for KAN

KANs are an alternative to traditional MLP layers. Unlike standard fully connected layers, KANs provide better approximation capabilities by leveraging mathematical functions derived from the Kolmogorov-Arnold representation theorem. This helps in:

- Reducing the number of required layers while maintaining accuracy.
- Enhancing interpretability compared to conventional deep learning models.
- Allowing more efficient training by leveraging structured function approximation.

2. Defining the KAN Layer in the Model

• Designing the KAN architecture:

- Unlike traditional fully connected layers with fixed activation functions, KAN layers involve adaptive basis functions.
- These basis functions are usually polynomials or trigonometric functions that help in approximating complex relationships in data.

• Defining the forward pass for the KAN layer:

- You implemented a function to transform input features using the KAN representation.
- This step included mapping inputs through a series of function transformations instead of simple weight multiplications as in MLPs.

3. Integrating the KAN Layer into the Model's Architecture

- Replacing the previous fully connected (dense) layers with the newly defined KAN layer.
- Ensuring that the KAN layer fits into the overall transformer structure while maintaining the input-output relationships.
- Adjusting forward and backward propagation mechanisms to correctly compute gradients and update weights.

4. Training and Evaluating the KAN-Based Model

- Same dataset partitions (training, validation, testing).
- Updated loss functions and optimization algorithms to ensure smooth training.
- Evaluation using previous metrics like Nash-Sutcliffe Efficiency (NSE), Evolution Criterion (Evol), and Total Percent Error (TPE) to compare performance with the original MLP-based model.

Converting MLP Layer to FAN (Fourier Activation Networks) Layer

Fourier Activation Networks (FANs) incorporate Fourier-based transformations in neural networks, improving function approximation by using periodic functions like sine and cosine instead of traditional activation functions like ReLU or Sigmoid.

Key Advantages of FANs

- **Better handling of high-frequency signals** – Useful in modeling complex, oscillatory data.
- **Improved generalization** – Reduces overfitting by providing smooth function approximations.
- **More expressive feature representations** – Captures intricate data patterns more efficiently than MLP layers.

1. Defining the FAN Layer in the Model

2. Integrating the FAN Layer into the Model Architecture

- Replacing fully connected (MLP) layers with the new FAN layer.
- Ensuring compatibility with the transformer structure, maintaining input-output dimensions.
- Updating the backpropagation mechanism to correctly compute gradients for Fourier-based transformations.

3. Training and Evaluating the FAN-Based Model

The model was then retrained with the FAN layer integrated, using the same approach as done in the KAN layer implementation.

Final Step: Dataset Tuning and Model Optimization

To ensure the model generalizes well and produces accurate predictions, I focused on minimizing loss and improving key evaluation metrics (NSE, Evol, TPE). The tuning strategies included:

1. Dataset Tuning: Minimizing Loss & Optimizing Metrics

a) Adjusting Learning Rate

- Lowering the learning rate prevents the model from making large weight updates, reducing instability.
- A smaller learning rate helps the model converge smoothly and avoid oscillations in loss.

b) Increasing Batch Size

- A larger batch size leads to more stable gradient updates, improving generalization.
- Helps in faster training by leveraging parallelism but may require more computational power.

c) Increasing the Number of Epochs

- More training epochs allow the model to learn deeper patterns in data.
- Helps in reducing the error margin and optimizing performance over multiple iterations.
- Required careful early stopping monitoring to prevent overfitting.

2. Model Optimization: Tuning Architectural Parameters

a) Changing Head Size

- The head size in multi-head attention determines how much information each attention head processes.
- Increasing the head size allows the model to capture more complex dependencies in data.

b) Adjusting ff_dim

- `ff_dim` refers to the dimensionality of the feed-forward layer in transformers.
- Larger `ff_dim` increases the model's capacity to learn non-linear transformations in data.

*c) Modifying the Number of Attention Heads (**num_heads**)*

- More attention heads enable better parallel processing and capture diverse relationships within data.
- However, excessive attention heads can lead to increased computation without significant gains.

d) Tuning Output Dimension

- The output dimension was adjusted based on the task-specific requirements, ensuring that predictions were mapped correctly to target outputs.
- Fine-tuning this parameter helped in aligning model outputs with expected evaluation metrics.

Project codes: [Google Drive Link](#)

By: Anisha Choudhary (IIT Kharagpur)